The Ch(e)at Code

Agent Patterns for Copilot Chat

ISSUE #016 • JULY 14, 2026 • 🧠 CONCEPTUAL PATTERN

You've heard this one before: "How do I know the agent is actually working?" The pilot shipped three weeks ago. Leadership wants ROI numbers. The agent owner wants quality metrics. Users want confidence that the answers are trustworthy. Nobody has any of these things — and nobody built the infrastructure to get them.

This issue's pattern: Agent Evaluation & Trust Signals — a three-layer framework for measuring whether your agent is adopted, accurate, and worth the investment, plus the user-facing signals that build confidence in every response.

🧩 PATTERN BREAKDOWN

Agent Evaluation & Trust Signals

Three evaluation layers + hallucination detection + regression testing + user trust signals

Why this matters: Every agent pilot hits a "prove it" moment at 30 days. Without evaluation infrastructure, the conversation becomes anecdotal: "Some people like it, some don't." This pattern gives you the toolkit to turn anecdotes into dashboards — and dashboards into expansion funding.

THREE EVALUATION LAYERS

```
Layer 1: Usage Metrics — Is anyone using it?
  Sessions/day, returning users, topic triggers, drop-off points
Layer 2: Quality Metrics — Are the answers good?
  Confidence scores, source grounding, hallucination rate, thumbs up/down
Layer 3: Business Metrics — Is it worth the investment?
```

```
  Tasks automated, escalations avoided, time-to-
resolution vs. baseline
```

The pattern has three design decisions:

1. **Layer your metrics, don't flatten them.** Usage tells you adoption. Quality tells you accuracy. Business tells you ROI. Each layer answers a different stakeholder's question. When you present to leadership, lead with business metrics. When you tune the agent, focus on quality. When you plan rollout, watch usage. Flattening these into a single "agent score" loses the signal you need to act.

2. **Detect hallucinations with layered signals, not a single check.** No single method catches all hallucinations. Stack them: confidence scoring (agent self-reports certainty — unreliable alone, useful as a signal), source grounding (did the answer cite a real document?), citation verification (does the cited passage actually support the claim?), and human spot-checks (random samples reviewed by SMEs). This is the Issue #005 Approval Gates pattern applied to quality assurance — human-in-the-loop as a quality layer, not just an approval layer.

3. **Build regression detection from day one.** Adding new documents to the knowledge source can silently degrade existing answers — the "new knowledge" trap. The pattern: maintain a *golden test set* of 20 known-good question-answer pairs. Run them weekly. Compare response similarity against your baseline. Alert when answers drift. This catches the problem that nobody notices until a user complains — and by then, trust is already eroded.

GOLDEN TEST SET FLOW

```
20 known-good Q&A pairs (stored in SharePoint list)
  ↓
Weekly Power Automate flow → sends each question to the
agent
  ↓
Compare response to expected answer (Azure OpenAI
similarity)
  ↓
Similarity drops below threshold?
  ↓
⚠ Teams alert: "3 of 20 golden answers degraded — review
changes"
```

Trust signals for end users: Evaluation metrics help *you* know the agent works. Trust signals help *users* believe it. These aren't cosmetic — they're the difference between users trusting the agent and users Googling instead.

- **Show confidence:** "I'm 85% confident based on 3 sources" — give the user a reason to trust the answer
- **Cite sources:** Link to the actual document, not just name it — verifiable citations change behavior
- **Flag uncertainty:** "I found partial information but couldn't fully answer — here's what I know and what's missing"
- **Enable feedback:** Thumbs up/down on every response — this closes the loop between user experience and quality metrics

Key insight: The golden test set is your *canary in the coal mine*. Most teams don't discover quality regressions until a user complains on Teams. By then, trust is already damaged. A weekly automated check on 20 known-good answers catches drift before users do — and gives the agent owner a concrete action: "Review the knowledge changes from this week."

On hallucination detection: Confidence scoring alone is unreliable — the model can be confidently wrong. Source grounding alone misses cases where the citation exists but doesn't support the claim. Citation verification alone is expensive at scale. **Stack all three plus human spot-checks** and you get a detection system where each layer covers the others' blind spots. This is defense-in-depth applied to accuracy.

⚠ **The "new knowledge" trap:** Adding documents to your knowledge source feels like improvement, but it can degrade existing answers. A new HR policy doc might cause the agent to blend old and new guidance. A duplicate document might split the agent's confidence. Without regression testing, you won't know until someone gets a wrong answer in production. Build the golden test set before you add new knowledge sources — not after.

🏆 **WHERE THIS PATTERN LANDS**

- **Any team 30 days post-pilot** — the "prove it" moment when leadership asks for ROI and the agent owner has nothing but anecdotes

- **Regulated industries (healthcare, finance, legal)** — where answer accuracy isn't a nice-to-have but a compliance requirement with real consequences
- **Multi-agent environments** — organizations running several agents where quality varies and there's no consistent way to compare performance across agents
- **Customer-facing agents** — where a hallucinated answer doesn't just waste time but damages brand trust and could create legal exposure
- **Expansion conversations** — when a successful pilot needs data to justify scaling from one team to the whole organization

Positioning tip: Don't lead with "you need monitoring." Lead with the question: "Right now, can you tell me how many people used your agent yesterday and whether the answers were good?" When the answer is no — and it almost always is — the conversation shifts from "why do I need this" to "how fast can we build it." Next issue (#017) builds the dashboard.

⚡ **QUICK TIPS**

✓ **Start with usage, not quality.** Usage metrics are free — Copilot Studio provides them out of the box. If nobody's using the agent, quality metrics are academic. Check session counts and returning user rates first. If adoption is low, fix the experience before you instrument quality.

✓ **20 golden questions is enough.** Don't over-engineer the test set. Pick the 20 most common or most critical questions users ask. Write the expected answers. Run them weekly. If your quality holds on 20 representative questions, it holds broadly. If it doesn't, you've caught the regression early.

✓ **Trust signals change user behavior.** Internal testing shows that agents displaying confidence scores and source links get 40–60% more repeat usage than agents that just show a bare answer. Users don't need perfect answers — they need to know *how much* to trust the answer they got.

✓ **Pair with Issue #005 (Approval Gates)** for human-in-the-loop quality checks. The Approval Gates pattern handles *pre-action* review. This pattern handles *post-deployment* evaluation. Together, they give you quality assurance at both ends of the agent lifecycle.

Next issue: We build the dashboard.

Issue #017 turns this framework into a working analytics stack — Copilot Studio + App Insights + Power BI + Power Automate.

The Ch(e)at Code

AIWF Team • AI Business Solutions

Questions or ideas for the next issue? Reply to this email.

Microsoft Internal • Not for external distribution